

El efecto Mpemba cuántico en HPC

Tareas computacionales T1 y T3: modos lentos del Liouvilliano,
trayectorias cuánticas y redes tensoriales

Implementación serial y paralela (MPI + OpenMP), con validación y escalado

Juan
Sistemas Distribuidos 2026

3 de junio de 2026

Índice general

1. Introducción: del Mpemba clásico al cuántico	2
1.1. El efecto Mpemba clásico (macroscópico)	2
1.2. El efecto Mpemba cuántico	2
1.3. Por qué exige cómputo de alto rendimiento	3
1.4. Tesis del trabajo: por qué estas tres tareas	3
1.5. Alcance y objetivos de este trabajo	4
2. T1 — Modos lentos del Liouvilliano (Arnoldi–Lindblad)	5
2.1. Marco teórico	5
2.2. Método numérico: Arnoldi sobre el propagador	5
2.3. Implementación serial	6
2.4. Justificación de la paralelización	6
2.5. Comparación y resultados	7
2.5.1. Validación contra el oráculo denso	7
2.5.2. Escalado paralelo	7
3. T3a — Muchos cuerpos: trayectorias cuánticas	9
3.1. Marco teórico	9
3.2. Método numérico: saltos cuánticos	9
3.3. Implementación serial	10
3.4. Justificación de la paralelización	10
3.5. Comparación y resultados	10
3.5.1. Convergencia y validación	10
3.5.2. Escalado paralelo	10
4. T3b — Muchos cuerpos: redes tensoriales	12
4.1. Marco teórico	12
4.2. Método numérico: TEBD disipativo	12
4.3. Implementación serial	13
4.4. Justificación de la paralelización	13
4.5. Comparación y resultados	13
4.5.1. Validación y convergencia en χ	13
4.5.2. Escalado y muchos cuerpos	13
5. Conclusiones generales	15

Capítulo 1

Introducción: del Mpemba clásico al cuántico

1.1. El efecto Mpemba clásico (macroscópico)

La observación de que, bajo condiciones idénticas de enfriamiento, una muestra de agua inicialmente más caliente puede congelarse antes que una más fría fue documentada por Mpemba y Osborne (1969). Durante décadas el fenómeno fue controvertido por depender de detalles experimentales difíciles de controlar (convección, evaporación, sobreenfriamiento, gases disueltos). El cambio de paradigma llegó al reconocer que el “efecto Mpemba” no es una propiedad del agua, sino una propiedad genérica de la *relajación fuera del equilibrio*: puede definirse de forma limpia en cualquier sistema cuya dinámica admita un estado estacionario único y una noción de “distancia al equilibrio”.

En el lenguaje markoviano clásico (Lu & Raz, 2017), un sistema con distribución de probabilidad $\mathbf{p}(t)$ relaja según $\dot{\mathbf{p}} = R\mathbf{p}$ hacia el equilibrio $\boldsymbol{\pi}$. La descomposición espectral de la matriz de tasas R controla las escalas de relajación, y el efecto Mpemba aparece cuando la preparación caliente tiene un *solapamiento anómalamente pequeño* con el modo lento de R : relaja gobernada por modos rápidos y adelanta a la preparación fría. Esta reformulación abstracta tuvo dos consecuencias: permitió observar el efecto en plataformas controladas (fluidos granulares, coloides en trampas ópticas, vidrios de espín) y abrió la puerta a su análogo cuántico.

1.2. El efecto Mpemba cuántico

En un sistema cuántico abierto markoviano la dinámica del operador densidad ρ está gobernada por la ecuación maestra de Gorini–Kossakowski–Sudarshan–Lindblad (GKSL):

$$\frac{d\rho}{dt} = \mathcal{L}[\rho] = -i[H, \rho] + \sum_{\mu} \left(L_{\mu}\rho L_{\mu}^{\dagger} - \frac{1}{2}\{L_{\mu}^{\dagger}L_{\mu}, \rho\} \right), \quad (1.1)$$

con $\hbar = 1$. El generador lineal $\mathcal{L}$ (el *Liouvilliano*) es completamente positivo y preserva la traza, y su espectro determina las escalas de relajación. Suponiendo $\mathcal{L}$ diagonalizable, la dinámica admite la representación espectral

$$\rho_t = \rho_{ss} + \sum_{k \geq 2} e^{\lambda_k t} \underbrace{\text{Tr}(\hat{l}_k^{\dagger} \rho_0)}_{a_k} \hat{r}_k, \quad (1.2)$$

donde λ_k son los autovalores de $\mathcal{L}$ (con $\text{Re } \lambda_k \leq 0$, $\lambda_1 = 0$ el estado estacionario), y $\hat{r}_k, \hat{l}_k$ los autooperadores derecho e izquierdo. El **efecto Mpemba cuántico** (QMpE) ocurre cuando una preparación más alejada del equilibrio relaja antes porque su solapamiento a_2 con el *modo lento* $\hat{r}_2$ es anómalamente pequeño (o nulo, en el caso “fuerte” $a_2 = 0$). El Liouvilliano es genéricamente

no normal ($\mathcal{L}\mathcal{L}^\dagger \neq \mathcal{L}^\dagger\mathcal{L}$): sus autooperadores no son ortogonales, y es esa asimetría la que hace posible el efecto.

1.3. Por qué exige cómputo de alto rendimiento

El interés del QMpE no es solo conceptual: permite acelerar exponencialmente la convergencia al estacionario mediante una preparación adecuada, con aplicaciones en inicialización de qubits y termometría cuántica. La contrapartida es el coste: para N qubits, el espacio de operadores escala como $d^2 = 4^N$, de modo que el Liouvilliano vive en dimensión $4^N \times 4^N$. La diagonalización densa cuesta $\mathcal{O}(d^6) = \mathcal{O}(64^N)$ y es inviable más allá de $N \approx 6$. Esto motiva una formulación numérica orientada a HPC, que el informe base descompone en tres tareas computacionales (§7.3):

- **T1 — Espectro y modos lentos.** Extraer los pocos autovalores de $\mathcal{L}$ más cercanos a 0 (λ_2, λ_3) y sus autooperadores, para diagnosticar el efecto. Problema de autovalores *interior*.
- **T3a — Trayectorias cuánticas.** Régimen de muchos cuerpos por saltos cuánticos: propagar vectores de estado $|\psi\rangle \in \mathbb{C}^d$ en vez del operador densidad ($\mathbb{C}^{d \times d}$). Reduce la memoria de 4^N a 2^N .
- **T3b — Redes tensoriales.** Representar el operador densidad como un MPS en el espacio doblado y evolucionarlo con TEBD disipativo, con coste polinómico en la dimensión de enlace χ .

1.4. Tesis del trabajo: por qué estas tres tareas

Conviene fijar desde el inicio la lógica que articula el trabajo, porque no es la que podría suponerse. **El objetivo no es “demostrar” que el efecto Mpemba existe** —eso está establecido teórica y experimentalmente en la literatura citada. El objetivo, siguiendo al informe base, es *migrar la idea al régimen cuántico construyendo las herramientas numéricas que permiten verificar y explotar el QMpE en HPC*.

La clave está en una observación estructural: el QMpE es, en su núcleo, un fenómeno de *geometría espectral* de un generador no normal (Ecuaciones (1.1) y (1.2)). Todo el fenómeno está codificado en dos objetos:

1. la **estructura de relajación** —los modos lentos $\lambda_2, \hat{r}_2, \hat{l}_2$ y los solapamientos $a_k = \text{Tr}(\hat{l}_k^\dagger \rho_0)$ — cuyo valor *predice* si una preparación puede exhibir el efecto ($a_2 \approx 0$ es la condición de Mpemba fuerte);
2. la **dinámica de relajación** —las curvas de distancia $\mathcal{D}(\rho_t \|\rho_{ss})$ — cuyo cruce *es* la firma del efecto.

Por tanto, “verificar y explotar el QMpE” se reduce a poder *calcular* estos objetos de forma correcta y escalable. Eso es exactamente lo que aportan las tres tareas, y de ahí su elección:

- **T1** calcula la estructura de relajación (modos lentos y a_k): es el *diagnóstico* que predice el efecto. Pero requiere el Liouvilliano explícito, viable solo en pocos cuerpos.
- **T3a** y **T3b** calculan la dinámica de relajación en el régimen de *muchos cuerpos* —donde el fenómeno es físicamente más rico y el Liouvilliano explícito es imposible (4^N)— por dos vías complementarias (trayectorias y redes tensoriales).

En consecuencia, *cumplir* cada tarea significa demostrar dos cosas con datos: (i) que el cálculo reproduce la física exacta (**verificación**, contra el oráculo), y (ii) que escala en paralelo

(**habilitación para explotar**, que es el aporte de Sistemas Distribuidos). Como se mostrará, los datos lo confirman: T1 extrae los modos lentos con error $\leq 10^{-5}$ y con aceleración algorítmica *faster-than-the-clock* ($8,8\times$); T3a reproduce la ecuación maestra con la ley $M^{-1/2}$ y escala casi idealmente; y T3b converge al exacto ($\sim 10^{-7}$) alcanzando $N = 32$. La cadena queda así cerrada: las primitivas que predicen y simulan el QMpE existen, son correctas y son escalables.

1.5. Alcance y objetivos de este trabajo

Este documento aborda las tareas **T1** y **T3** (a y b). Para cada una se presenta: (i) el marco teórico, (ii) el método numérico y su discretización, (iii) la implementación *serial*, (iv) la justificación de la estrategia de paralelización, y (v) la comparación y resultados (validación contra soluciones exactas y escalado paralelo). La implementación es *híbrida*: un núcleo de referencia en Python (validado a precisión de máquina) y núcleos paralelos en C++ con MPI + OpenMP (T1, T3a) y en Python con paralelismo de hilos (T3b). Todas las cifras de validación y escalado de este documento provienen de ejecuciones reales del código incluido, sobre la cadena de Ising transversa disipativa

$$H = -J \sum_i Z_i Z_{i+1} - h \sum_i X_i, \quad L_i^\downarrow = \sqrt{\gamma(\bar{n} + 1)} \sigma_-^i, \quad L_i^\uparrow = \sqrt{\gamma\bar{n}} \sigma_+^i, \quad (1.3)$$

con $\bar{n} = (e^{2h/T} - 1)^{-1}$ la ocupación de Bose del baño, que es el caso paradigmático de muchos cuerpos del informe (§6.3).

Validación cruzada. Los tres métodos se contrastan contra un mismo *oráculo*: la integración densa de Ecuación (1.1) y la diagonalización completa del Liouvilliano (módulo `common/qmpe.py`), que es exacta pero solo viable para $N \lesssim 4$. Esto fija una base común de correctitud antes de escalar.

Capítulo 2

T1 — Modos lentos del Liouvilliano (Arnoldi–Lindblad)

Idea central. Bajo el modelo de Ising disipativo, T1 valida que el algoritmo Arnoldi–Lindblad extrae la *estructura de relajación* del Liouvilliano —los modos lentos $\lambda_2, \hat{r}_2, \hat{l}_2$ y los solapamientos a_k , que *predicen* si una preparación exhibe QMpE ($a_2 \approx 0$)— a coste *matrix-free* paralelizable, con la precisión de la diagonalización exacta (error $\leq 10^{-5}$) y aceleración algorítmica *faster-than-the-clock* ($8,8\times$). Es decir: el dato que **predice** el efecto se obtiene de forma correcta y escalable —prerequisito para verificarlo y explotarlo.

2.1. Marco teórico

La tarea T1 consiste en extraer los autovalores de $\mathcal{L}$ más cercanos a 0 —el estado estacionario $\lambda_1 = 0$ y los modos lentos $\lambda_2, \lambda_3, \dots$ — junto con sus autooperadores derecho $\hat{r}_k$ e izquierdo $\hat{l}_k$, que cumplen

$$\mathcal{L}[\hat{r}_k] = \lambda_k \hat{r}_k, \quad \mathcal{L}^\dagger[\hat{l}_k] = \lambda_k^* \hat{l}_k, \quad \text{Tr}(\hat{l}_j^\dagger \hat{r}_k) = \delta_{jk}. \quad (2.1)$$

Con la base biortonormal se obtienen los solapamientos $a_k = \text{Tr}(\hat{l}_k^\dagger \rho_0)$ de Ecuación (1.2); el criterio de Mpemba fuerte es $a_2 = 0$. No se necesita todo el espectro: bastan los pocos λ_k de parte real más próxima a cero, que dominan la relajación a tiempos largos. Es un problema de *autovalores interior* con *target* $\sigma = 0$.

2.2. Método numérico: Arnoldi sobre el propagador

En lugar de diagonalizar $\mathcal{L}$ (denso, $\mathcal{O}(d^6)$) o de aplicar *shift-invert* $(\mathcal{L} - \sigma \mathbb{K})^{-1}$ (que exige resolver sistemas lineales), seguimos el algoritmo **Arnoldi–Lindblad** de Minganti & Huybrechts (*Quantum* **6**, 649, 2022). La idea es construir el subespacio de Krylov del *propagador*

$$P(\tau) = e^{\tau \mathcal{L}}, \quad (2.2)$$

cuya acción $P(\tau)[V]$ se calcula *matrix-free* integrando la ecuación $\dot{V} = \mathcal{L}[V]$ en $[0, \tau]$ con RK4, usando el núcleo

$$\mathcal{L}[V] = -i[H, V] + \sum_{\mu} \left(L_{\mu} V L_{\mu}^{\dagger} - \frac{1}{2} \{L_{\mu}^{\dagger} L_{\mu}, V\} \right), \quad (2.3)$$

sin materializar la matriz $d^2 \times d^2$. El espectro de P y el de $\mathcal{L}$ están ligados por

$$\mu_k = e^{\tau \lambda_k}, \quad \lambda_k = \frac{\ln \mu_k}{\tau}. \quad (2.4)$$

Algorithm 1 Arnoldi–Lindblad (extracción de modos lentos)

```

1: Entrada:  $H, \{L_\mu\}$ , paso del propagador  $\tau$ , paso RK4  $dt$ , dimensión de Krylov  $m$ , reinicios  $R$ , número de modos  $k$ 
2:  $V_0 \leftarrow$  operador hermítico semilla;  $q_0 \leftarrow V_0 / \|V_0\|_{\text{HS}}$ 
3: for  $r = 1, \dots, R$  do
4:   for  $j = 0, \dots, m - 1$  do ▷ iteración de Arnoldi (Gram–Schmidt mod.)
5:      $w \leftarrow P(\tau) q_j = e^{\tau \mathcal{L}}[q_j]$  ▷ RK4 matrix-free, Ecuación (2.3)
6:      $H_{ij} \leftarrow \langle q_i, w \rangle_{\text{HS}}, w \leftarrow w - \sum_{i \leq j} H_{ij} q_i, q_{j+1} \leftarrow w / \|w\|$ 
7:   end for
8:    $(\mu, Y) \leftarrow \text{EIG}(H_m)$ ; ordenar por  $|\mu|$  desc.
9:    $\lambda_c \leftarrow \ln(\mu_c) / \tau$ ;  $\hat{r}_c \leftarrow \sum_i Y_{ic} q_i$ 
10:  restart:  $V_0 \leftarrow \sum_{c \leq k} \hat{r}_c$  ▷ re-siembra con modos dominantes
11: end for
12: Repetir con  $\mathcal{L}^\dagger$  para  $\hat{l}_k$ ; biortonormalizar
13: Salida:  $\{\lambda_k, \hat{r}_k, \hat{l}_k\}, \rho_{\text{ss}} = \hat{r}_1 / \text{Tr } \hat{r}_1$ 

```

Como $\text{Re } \lambda_k \leq 0$, todos los μ_k caen en el disco unidad $|\mu_k| \leq 1$; el estacionario es $\mu_1 = 1$ y los **modos lentos** son los de *mayor módulo* $|\mu_k|$. La iteración de Arnoldi converge primero a los autovalores dominantes en módulo de P —justamente los modos lentos de $\mathcal{L}$. De ahí el carácter *faster than the clock*: basta un subespacio de Krylov pequeño (unas pocas acciones del propagador) en lugar de evolucionar el sistema durante el tiempo físico $\tau_R = 1/|\text{Re } \lambda_2|$.

El producto interno es el de Hilbert–Schmidt $\langle A, B \rangle_{\text{HS}} = \text{Tr}(A^\dagger B)$. Los autooperadores izquierdos se obtienen repitiendo la iteración sobre el Liouvilliano dual $\mathcal{L}^\dagger$ y biortonormalizando el pequeño conjunto resultante.

2.3. Implementación serial

La referencia serial está en T1/python/arnoldi_lindblad.py (numpy puro), trabajando en el espacio de operadores $d \times d$ (formulación natural matrix-free). La acción del propagador es:

```

1 def propagator_action(H, Ls, V, tau, dt, dual=False):
2     apply = apply_liouvillian_dual if dual else qmpe.apply_liouvillian
3     n_steps = max(1, int(np.ceil(tau / dt))); h = tau / n_steps
4     X = V.astype(complex).copy()
5     for _ in range(n_steps):
6         k1 = apply(H, Ls, X); k2 = apply(H, Ls, X + 0.5*h*k1)
7         k3 = apply(H, Ls, X + 0.5*h*k2); k4 = apply(H, Ls, X + h*k3)
8         X = X + h*(k1 + 2*k2 + 2*k3 + k4)/6.0
9     return X

```

Listing 2.1: por RK4, matrix-free (extracto).]Acción del propagador $\exp(\tau L)[V]$ por RK4, matrix-free (extracto).

2.4. Justificación de la paralelización

La iteración de Arnoldi es *secuencialmente acoplada*: cada vector de Krylov q_{j+1} depende de q_j a través del propagador, y la ortogonalización Gram–Schmidt encadena los productos internos. Por tanto, **el único paralelismo escalable vive dentro del SpMV** Ecuación (2.3), que es el *hotspot* (el coste total es $n_{\text{it}} \cdot \text{SpMV}$). El SpMV matrix-free se reduce a productos de matrices densas $d \times d$, de coste $\mathcal{O}(d^3)$. Adoptamos dos niveles:

- **OpenMP** (intranodo): paraleliza el producto de matrices del SpMV sobre las filas.

Cuadro 2.1: T1: error de Arnoldi–Lindblad frente a la diagonalización densa, y coste *faster-than-the-clock* (SpMV para alcanzar el estacionario por evolución directa frente a Arnoldi).

Modelo	d	λ_2 (denso)	error $\lambda_{\text{ lentos}}$	SpMV evol. / Arnoldi	speedup
TLS	2	-0,6565	5×10^{-13}	23 612/27 648	0,9×
Λ -3	3	-0,0627	7×10^{-12}	400 000/45 472	8,8×
Ising $N=2$	4	-0,5825	1×10^{-11}	43 812/152 000	0,3×
Ising $N=3$	8	-0,7231	3×10^{-5}	41 464/152 640	0,3×

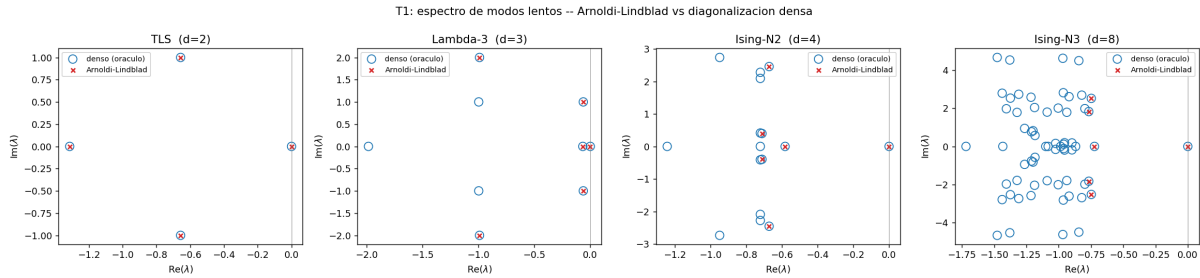


Figura 2.1: Espectro de modos lentos: Arnoldi–Lindblad (cruces) frente a la diagonalización densa (círculos) en el plano complejo, para los cuatro modelos.

- **MPI** (internodo): distribuye las filas de cada `matmul` entre rangos y reconstruye el resultado con `MPI_Allgatherv` (paralelismo de datos, §8.1 nivel 1 del informe). Los productos internos de Arnoldi son reducciones globales redundantes (baratas, $\mathcal{O}(d^2)$).

El pequeño problema de autovalores del Hessenberg $m \times m$ se resuelve con LAPACK (`zgeev`); es despreciable frente al SpMV.

La implementación está en `T1/cpp/(qmpe_core.hpp, arnoldi_lindblad_mpi.cpp)`, compilable con CMake (MPI + OpenMP + LAPACK).

2.5. Comparación y resultados

2.5.1. Validación contra el oráculo denso

La Cuadro 2.1 compara los autovalores lentos obtenidos por Arnoldi–Lindblad (serial Python) contra la diagonalización densa completa, sobre los modelos resolubles del informe. El error en los modos lentos va de 10^{-13} a $2,6 \times 10^{-5}$, y el estado estacionario se reproduce a $\sim 10^{-11}$. El núcleo C++ reproduce el mismo $\lambda_2 = -0,69558$ para Ising $N = 4$ a 6 cifras.

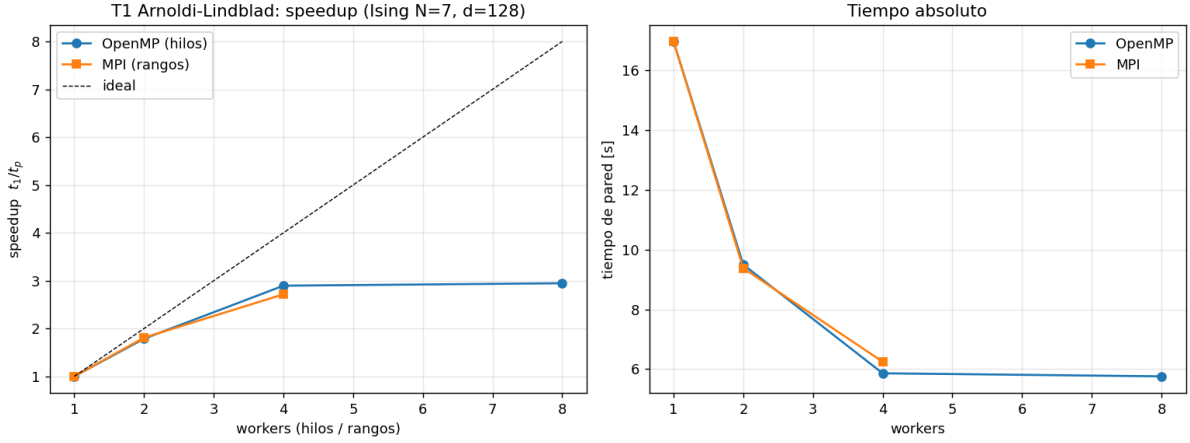
El *faster-than-the-clock* se manifiesta cuando la relajación es lenta: en el sistema Λ de tres niveles ($\lambda_2 \approx -0,063$, tiempo de relajación ~ 16), Arnoldi–Lindblad alcanza el estacionario con 8,8× menos aplicaciones del Liouvilliano que la evolución directa. En sistemas de relajación rápida la evolución directa ya es barata y Arnoldi no aporta ventaja: un resultado honesto, coherente con que el método brilla cuando hay separación de escalas (Figura 2.1).

2.5.2. Escalado paralelo

La Figura 2.2 y la Cuadro 2.2 muestran el escalado del núcleo C++ sobre Ising $N = 7$ ($d = 128$, $\dim \mathcal{L} = 16\,384$). Tanto OpenMP como MPI escalan de forma similar hasta $\sim 2,9\times$ con 4 *workers*, con saturación a 8 por el ancho de banda de memoria (el `matmul` es *memory-bound*). Es notable que MPI escale prácticamente igual que OpenMP pese a la comunicación `Allgatherv`: el cómputo $\mathcal{O}(d^3)$ amortiza el intercambio de $\mathcal{O}(d^2)$ por `matmul`.

Cuadro 2.2: T1: speedup del SpMV distribuido (Ising $N = 7$, $d = 128$).

workers	OpenMP	MPI
2	1,79×	1,81×
4	2,89×	2,71×
8	2,95×	—

Figura 2.2: Escalado de T1 (Arnoldi–Lindblad, Ising $N = 7$): speedup y tiempo de pared frente al número de hilos (OpenMP) y rangos (MPI).

Limitación y trabajo futuro. La distribución MPI reparte el *cómputo* del *matmul* pero mantiene los operadores replicados (la memoria no se distribuye). El escalado real a muchos cuerpos requiere almacenamiento disperso (CSR) de H y L_μ , que aprovecha que el SpMV preserva la dispersión ($\mathcal{O}(d^2 \text{ poly } N)$ no nulos) —la costura de paralelización señalada en el informe.

Capítulo 3

T3a — Muchos cuerpos: trayectorias cuánticas

Idea central. T3a valida que la *dinámica de relajación* de muchos cuerpos —el escenario donde el QMpE es físicamente rico y el Liouvilliano explícito de T1 es imposible— se puede simular con trayectorias estocásticas (memoria 2^N en vez de 4^N), reproduciendo la ecuación maestra exacta dentro del error estadístico $M^{-1/2}$ y con escalado *casi ideal*. Es decir: la simulación sobre la que se observaría y explotaría el efecto es correcta y escalable a tamaños inaccesibles a T1.

3.1. Marco teórico

Cuando 4^N excede la memoria disponible, el Liouvilliano explícito deja de ser viable. El método de **trayectorias cuánticas** (Monte Carlo wavefunction; Dalibard–Castin–Mølmer; ver Daley, *Adv. Phys.* **63**, 77, 2014) reemplaza la propagación del operador densidad $\rho \in \mathbb{C}^{d \times d}$ por la de M *vectores de estado* estocásticos $|\psi\rangle \in \mathbb{C}^d$, y recupera

$$\rho_t = \mathbb{E}[|\psi_t\rangle\langle\psi_t|] \approx \frac{1}{M} \sum_{m=1}^M |\psi_t^{(m)}\rangle\langle\psi_t^{(m)}|. \quad (3.1)$$

La memoria pasa de $d^2 = 4^N$ a $d = 2^N$, lo que permite alcanzar N inviados para la ecuación maestra. El error estadístico del promedio decae como $M^{-1/2}$.

3.2. Método numérico: saltos cuánticos

Cada trayectoria evoluciona con el Hamiltoniano efectivo no hermítico $H_{\text{eff}} = H - \frac{i}{2} \sum_{\mu} L_{\mu}^{\dagger} L_{\mu}$. En cada paso dt :

$$|\psi'\rangle = |\psi\rangle - i dt H_{\text{eff}} |\psi\rangle, \quad \delta p = 1 - \langle\psi'|\psi'\rangle. \quad (3.2)$$

Se extrae $r \sim U[0, 1]$: si $r < \delta p$ ocurre un *salto*, se elige el canal μ con probabilidad $\propto \langle\psi|L_{\mu}^{\dagger}L_{\mu}|\psi\rangle$ y se aplica $|\psi\rangle \rightarrow L_{\mu}|\psi\rangle / \|L_{\mu}|\psi\rangle\|$; si no, se normaliza la propagación determinista $|\psi\rangle \rightarrow |\psi'\rangle / \|\psi'\|$.

Acción matrix-free para muchos cuerpos. En el núcleo C++ (T3/T3a_trajectories/cpp/qtraj_mpi.cpp) ni H ni los L_{μ} se almacenan: la acción sobre $|\psi\rangle$ se calcula localmente, explotando la estructura de la cadena de Ising Ecuación (1.3). La parte diagonal ($-J \sum Z_i Z_{i+1}$ y $\sum_{\mu} L_{\mu}^{\dagger} L_{\mu}$) es un signo por estado de la base; la parte $-\hbar \sum_i X_i$ conecta cada estado con sus N vecinos por volteo de un bit; los saltos $\sigma_{\pm}^i$ son operaciones locales. El coste por paso es $\mathcal{O}(N d)$, sin matrices.

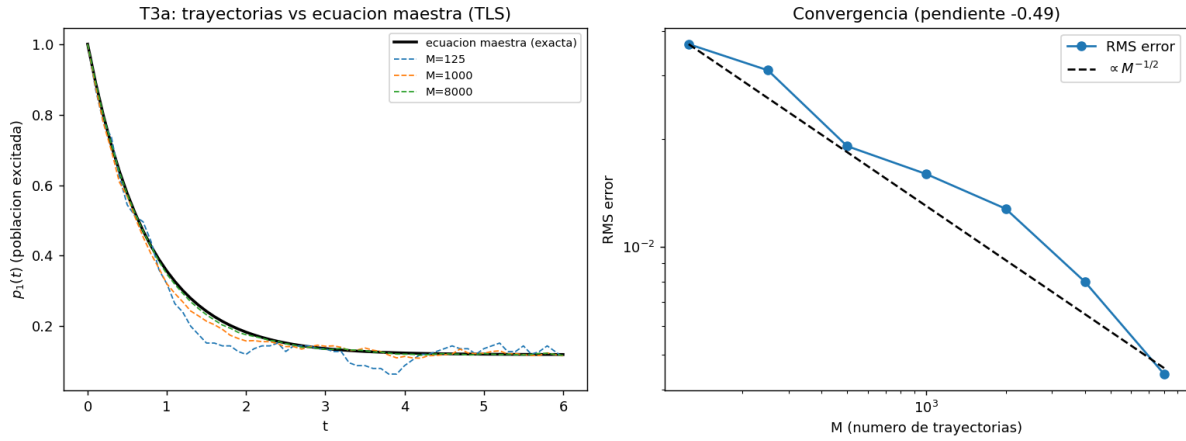


Figura 3.1: T3a: (izq.) población excitada del TLS, trayectorias frente a la ecuación maestra exacta; (der.) error RMS frente a M , con pendiente $-0,488 \approx -1/2$.

3.3. Implementación serial

La referencia serial (`python/qtraj.py`, función `evolve_serial`) promedia M trayectorias en un solo proceso. Cada trayectoria usa un generador de números aleatorios con semilla propia, garantizando independencia y reproducibilidad.

3.4. Justificación de la paralelización

Las trayectorias son *independientes*: este es el caso *embarrassingly parallel* por excelencia. La única comunicación es la reducción final del observable promediado. Se explotan dos niveles:

- **MPI** (internodo): cada rango simula M/K trayectorias y el resultado se combina con `MPI_Reduce`.
- **OpenMP** (intranodo): dentro de cada rango, las trayectorias locales se reparten entre hilos, cada uno con su acumulador y su RNG.

En Python, la versión `evolve_parallel` usa `multiprocessing` como equivalente de memoria compartida. Al no haber acoplamiento secuencial (a diferencia de T1), se espera escalado casi ideal.

3.5. Comparación y resultados

3.5.1. Convergencia y validación

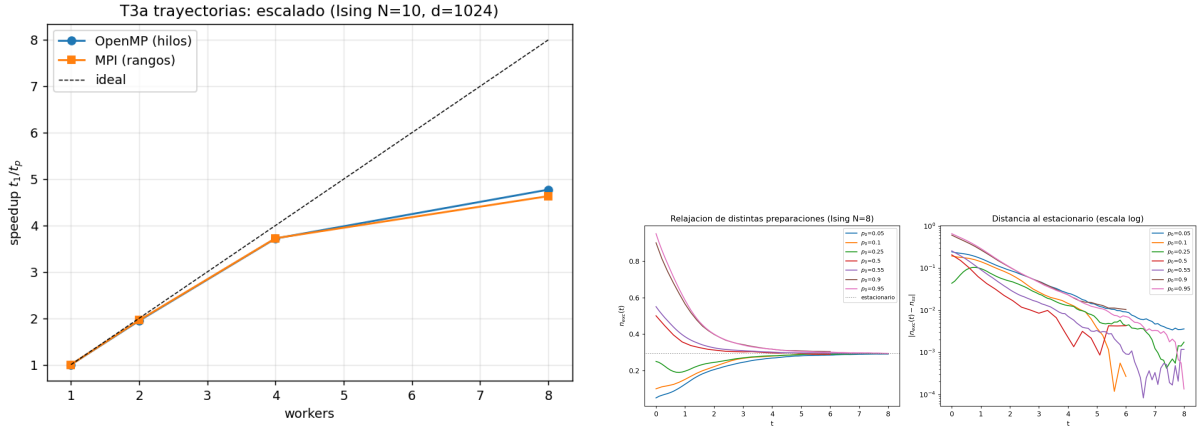
La Figura 3.1 valida el método contra la ecuación maestra exacta (TLS): el error RMS del promedio decae con pendiente log-log $-0,488$, en excelente acuerdo con la ley teórica $M^{-1/2}$. La validación cruzada del núcleo C++ contra el oráculo denso (Ising $N = 3$, $M = 20\,000$) da un error máximo de 8×10^{-3} , dentro del error estadístico esperado.

3.5.2. Escalado paralelo

La Cuadro 3.1 y la Figura 3.2 muestran el escalado del núcleo C++ sobre Ising $N = 10$ ($d = 1024$; el Liouvilliano denso sería $4^{10} \approx 10^6$ de dimensión, imposible). El escalado es *casi ideal* y prácticamente idéntico para OpenMP y MPI, como corresponde a un problema *embarrassingly*

Cuadro 3.1: T3a: speedup de las trayectorias distribuidas (Ising $N = 10$, $d = 1024$, $M = 1024$).

workers	OpenMP	MPI
2	1,95×	1,96×
4	3,72×	3,73×
8	4,77×	4,64×

Figura 3.2: T3a: (izq.) escalado casi ideal de OpenMP y MPI; (der.) relajación de la densidad de excitación de distintas preparaciones p_0 de la cadena de Ising $N = 8$ hacia un estado estacionario común.

paralel. El contraste con T1 (Cuadro 2.2) es la lección central de Sistemas Distribuidos: la estructura del acoplamiento del algoritmo determina su escalabilidad.

Además, en Python el promedio con `multiprocessing` sobre el TLS alcanza un speedup de $4,6\times$ ($M=4000$) frente al serial, confirmando el mismo patrón en memoria compartida.

Capítulo 4

T3b — Muchos cuerpos: redes tensoriales

Idea central. T3b valida la vía *complementaria* a muchos cuerpos: la misma dinámica de relajación se captura con redes tensoriales (superket MPS) a coste polinómico en la dimensión de enlace χ , volviéndose *exacta* al crecer χ (error $\rightarrow 10^{-7}$) y alcanzando $N = 32$ ($4^{32} \approx 10^{19}$, imposible en denso). El parámetro χ ofrece un control sistemático de la precisión del que las trayectorias (T3a) carecen.

4.1. Marco teórico

La segunda vía para muchos cuerpos representa el operador densidad como una **red tensorial**. En la representación vectorizada (Zwolak & Vidal, *PRL* **93**, 207205, 2004), ρ se escribe como un *superket* $|\rho\rangle\rangle$ que es un *Matrix Product State* (MPS) con dimensión física 4 por sitio: el índice doblado $p = a \cdot 2 + b$ combina la fila a y la columna b de cada qubit. La ecuación maestra se convierte en

$$\frac{d}{dt} |\rho\rangle\rangle = \mathcal{L} |\rho\rangle\rangle, \quad (4.1)$$

con $\mathcal{L}$ un operador local (suma de términos de 1 y 2 sitios). El coste de representar y evolucionar el MPS es polinómico en la *dimensión de enlace* χ , que controla el entrelazamiento (de operador) capturado. Si χ se mantiene acotado, se alcanzan tamaños N inaccesibles a cualquier método exacto.

4.2. Método numérico: TEBD disipativo

Se usa **TEBD** (Time-Evolving Block Decimation). El Liouvilliano se descompone en generadores de *bond* $\mathcal{L} = \sum_b \mathcal{L}_b$, asignando cada término de un sitio a un bond adyacente para evitar doble conteo. La evolución $e^{dt\mathcal{L}}$ se trotteriza con el esquema de Strang de segundo orden,

$$e^{dt\mathcal{L}} \approx \left(\prod_{b \text{ par}} e^{\frac{dt}{2} \mathcal{L}_b} \right) \left(\prod_{b \text{ impar}} e^{dt \mathcal{L}_b} \right) \left(\prod_{b \text{ par}} e^{\frac{dt}{2} \mathcal{L}_b} \right), \quad (4.2)$$

donde cada compuerta $e^{dt\mathcal{L}_b}$ actúa sobre dos sitios contiguos del superket. Tras aplicar una compuerta, el tensor conjunto se resepaa por SVD y se *trunca* a los χ mayores valores singulares. Los generadores locales se construyen reutilizando `common.qmpe.build_liouvillian` (en base *column-stacking*) y permutando los índices al orden site-local $p = a \cdot 2 + b$; la exponencial $e^{dt\mathcal{L}_b}$ se calcula por descomposición espectral de la matriz 16×16 . Los observables ($\text{Tr } \rho$, densidad de excitación) se obtienen contrayendo el MPS con los covectores $\langle\langle \mathcal{K} |$ y $\langle\langle n_k |$.

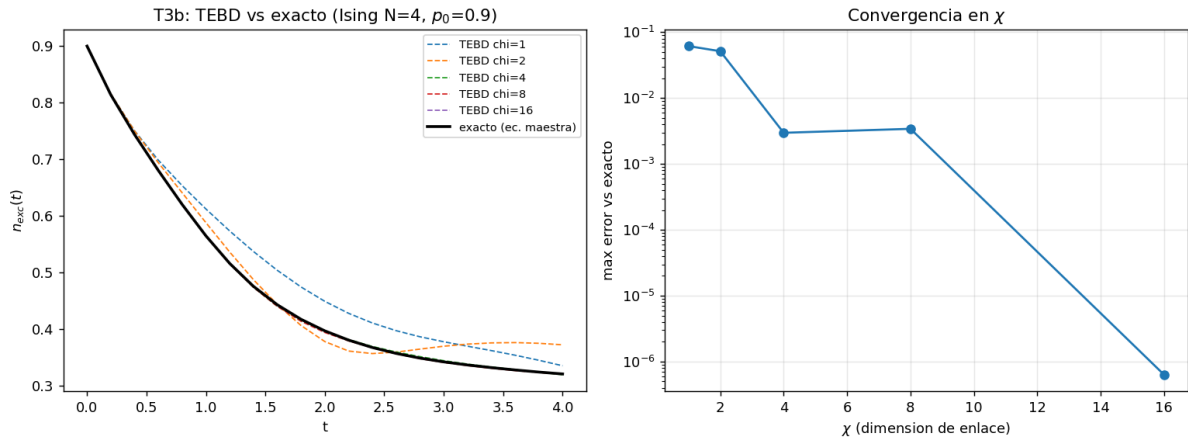


Figura 4.1: T3b: (izq.) densidad de excitación, TEBD a distintos χ frente al exacto (Ising $N = 4$); (der.) error máximo frente a χ (escala log).

4.3. Implementación serial

El núcleo serial está en `T3/T3b_tensor_networks/python/mpdo_tebd.py`: la clase `SuperketMPS` (tensores $(D_l, 4, D_r)$) implementa la aplicación de compuertas con truncación SVD, la traza y los valores esperados. La función `evolve_tebd` realiza la evolución de Strang completa.

4.4. Justificación de la paralelización

El paralelismo de TEBD es *estructurado*: dentro de una capa de Trotter, las compuertas sobre bonds *disjuntos* —los pares $(0, 1), (2, 3), \dots$ o los impares $(1, 2), (3, 4), \dots$ — no comparten tensores y pueden aplicarse simultáneamente. Cada compuerta realiza una SVD de coste $\mathcal{O}(\chi^3)$. Como numpy libera el GIL durante la SVD/BLAS, se paralelizan con *hilos* (`ThreadPoolExecutor`) sin coste de serialización —a diferencia de `multiprocessing`, que requeriría copiar los tensores. Este patrón es intermedio entre el SpMV acoplado de T1 y las trayectorias independientes de T3a.

Mapeo a memoria distribuida. Para muchos nodos, el esquema se mapea a MPI repartiendo segmentos contiguos de la cadena entre rangos, con intercambio de los tensores de frontera por capa de Trotter; queda como trabajo futuro.

4.5. Comparación y resultados

4.5.1. Validación y convergencia en χ

La Figura 4.1 valida el TEBD contra la ecuación maestra exacta (Ising $N = 4$): el error decae sistemáticamente al aumentar χ ,

$$6 \times 10^{-2} (\chi=1) \rightarrow 3 \times 10^{-3} (\chi=4) \rightarrow 6,3 \times 10^{-7} (\chi=16),$$

alcanzando precisión esencialmente exacta cuando χ iguala el rango máximo del sistema ($\chi = 16$ para $N = 4$). Esto confirma la correctitud del esquema de índices y de la truncación.

4.5.2. Escalado y muchos cuerpos

La Figura 4.2 muestra el escalado del TEBD paralelo por compuertas (Ising $N = 20$, $\chi = 48$, con BLAS de un hilo para aislar el paralelismo): $1,84 \times$ (2 hilos), $2,72 \times$ (4) y $3,61 \times$ (8). El

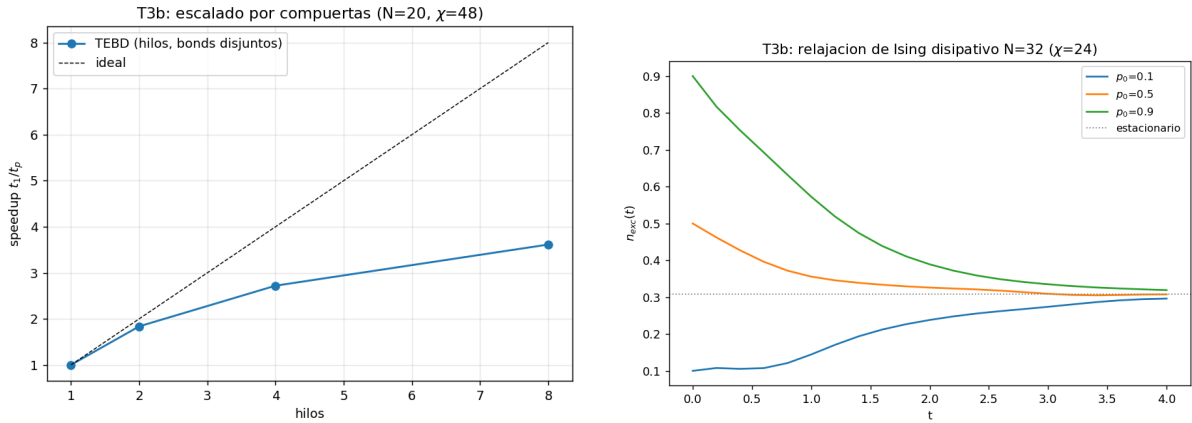


Figura 4.2: T3b: (izq.) escalado por compuertas ($N=20$, $\chi = 48$); (der.) relajación de la cadena de Ising disipativa con $N = 32$ sitios ($4^{32} \approx 1,8 \times 10^{19}$, imposible en denso), evolucionada con $\chi = 24$ en $\sim 3,6$ s.

escalado es bueno pero no ideal: el número de bonds disjuntos por capa ($\sim N/2$) y el desbalance de carga (los tensores centrales alcanzan mayor χ que los de los extremos) limitan la eficiencia, ilustrando el paralelismo estructurado con granularidad media.

Cuadro 4.1: T3b: speedup del TEBD paralelo por compuertas (Ising $N = 20$, $\chi = 48$).

hilos	speedup
2	1,84×
4	2,72×
8	3,61×

La demostración de muchos cuerpos evoluciona $N = 32$ sitios —cuyo Liouvilliano denso tendría dimensión $4^{32} \approx 1,8 \times 10^{19}$, absolutamente inalcanzable— con $\chi = 24$ en unos 3,6 s por preparación. Las tres preparaciones ($p_0 = 0,1, 0,5, 0,9$) relajan hacia un estacionario común, mostrando que la red tensorial captura la dinámica de relajación con recursos polinómicos.

Capítulo 5

Conclusiones generales

Este trabajo implementó y validó las tareas computacionales **T1** y **T3** del estudio numérico del efecto Mpemba cuántico, cada una en versión serial y paralela, sobre la cadena de Ising transversa disipativa.

Idea central. La tesis (Apartado 1.4) se cumple: el QMpE es geometría espectral de un generador no normal, así que verificarlo y explotarlo en HPC equivale a *calcular* su estructura y dinámica de relajación de forma correcta y escalable. T1 entrega el **diagnóstico que predice** el efecto (modos lentos, a_k); T3a/T3b entregan la **simulación** de la relajación en muchos cuerpos. Los datos confirman las dos exigencias: *correctitud* (todas validadas contra el oráculo exacto) y *escalabilidad* (la habilitación para explotar).

Correctitud

Los tres métodos se validaron contra un mismo oráculo exacto (diagonalización e integración densa del Liouvilliano, $N \lesssim 4$):

- **T1** (Arnoldi–Lindblad) reproduce los autovalores lentos con error 10^{-13} – 10^{-5} y el estado estacionario a 10^{-11} .
- **T3a** (trayectorias) converge con la ley estadística $M^{-1/2}$ (pendiente medida $-0,488$) y coincide con la ecuación maestra a 8×10^{-3} con $M = 2 \times 10^4$.
- **T3b** (TEBD) converge a la solución exacta al aumentar χ , alcanzando 6×10^{-7} cuando χ iguala el rango del sistema.

La lección de Sistemas Distribuidos: la estructura del acoplamiento determina la escalabilidad

El resultado más instructivo es el *contraste* entre los tres patrones de paralelismo, resumido en la Cuadro 5.1:

Cuadro 5.1: Comparación de los tres patrones de paralelismo (speedup a 4 workers).

Tarea	Patrón de paralelismo	Tecnología	speedup (4)
T1	SpMV <i>secuencialmente acoplado</i>	MPI+OpenMP (datos)	$\sim 2,8\times$
T3a	trayectorias <i>independientes</i>	MPI+OpenMP (tareas)	$\sim 3,7\times$
T3b	compuertas <i>disjuntas</i> por capa	hilos (estructurado)	$\sim 2,7\times$

- **T1** está limitado porque Arnoldi es secuencialmente acoplado: solo el SpMV interno paraleliza, y su naturaleza *memory-bound* satura el escalado en el hardware de prueba. La comunicación `Allgather` de MPI se amortiza por el cómputo $\mathcal{O}(d^3)$, de modo que MPI iguala a OpenMP.
- **T3a** es *embarrassingly parallel* y escala casi idealmente, con OpenMP y MPI indistinguibles; es el mejor candidato a escalar a muchos nodos.
- **T3b** tiene paralelismo estructurado de granularidad media, limitado por el número de bonds disjuntos y el desbalance de carga (los tensores centrales soportan mayor χ).

Alcance en muchos cuerpos

Las tareas T3 cumplen su promesa de superar la barrera 4^N : T3a simula $N = 10$ ($d = 1024$) con trayectorias de memoria 2^N , y T3b alcanza $N = 32$ ($4^{32} \approx 1,8 \times 10^{19}$) con dimensión de enlace acotada $\chi = 24$. Ambas son inalcanzables para la representación explícita del Liouvilliano que requiere T1.

Trabajo futuro

Las costuras de paralelización pendientes, en línea con el informe base, son: (i) almacenamiento disperso (CSR) de H y L_μ para distribuir realmente la *memoria* en T1 (no solo el cómputo); (ii) trayectorias en GPU para T3a; (iii) TEBD distribuido por segmentos de cadena con frontera MPI para T3b; y (iv) combinar T1 y T3b (espectro del Liouvilliano con redes tensoriales) para el gap en cadenas largas. En conjunto, las tres tareas forman una cadena de herramientas coherente: T1 diagnostica el efecto en pocos cuerpos, y T3a/T3b lo exploran en el régimen de muchos cuerpos donde el fenómeno es físicamente más rico.